

阶段 2：监督微调 (Supervised Fine-tuning)

Shuhan Tan¹, Kairan Dou², Yue Zhao¹, Philipp Krähenbühl¹

¹UT Austin

²Nankai University

Code & Model: https://ariostgx.github.io/ript_vla/

2026 年 8 月 17 日

1 Vision-Language-Action 模型的交互式后训练

Vision-Language-Action 模型的交互式后训练

摘要

我们提出 RIPT-VLA：一种简单且可扩展的、基于强化学习的交互式后训练范式。该方法仅依赖稀疏的二值成功奖励，即可对预训练 Vision-Language-Action (VLA) 模型进行微调 (fine-tuning)。现有的 VLA 训练流程高度依赖离线 expert demonstration 数据与监督式模仿学习，因此在低数据条件下，面对新任务和新环境时适应能力受限。RIPT-VLA 通过一种稳定的 policy 优化算法解决了这一问题；该算法基于动态 rollout sampling 与 leave-one-out advantage estimation，从而支持交互式后训练。

RIPT-VLA 具有以下特点。第一，它适用于多种 VLA 模型：在轻量级 QueST 模型上带来 21.2% 的性能提升，并将 7B 的 OpenVLA-OFT 模型提升到前所未有的 97.5% 成功率 (success rate)。第二，RIPT-VLA 同时具备较高的计算效率与数据效率：仅使用一个 demonstration，RIPT-VLA 就能在 15 次迭代内，将一个原本几乎不可用的 SFT 模型 (4%) 提升到 97% 的成功率。此外，我们还表明，RIPT-VLA 学到的 policy 能够泛化到不同任务与场景，并且对初始状态上下文具有良好的鲁棒性。这些结果表明，RIPT-VLA 是一种在极少监督条件下对 VLA 模型进行后训练的实用且有效的范式。

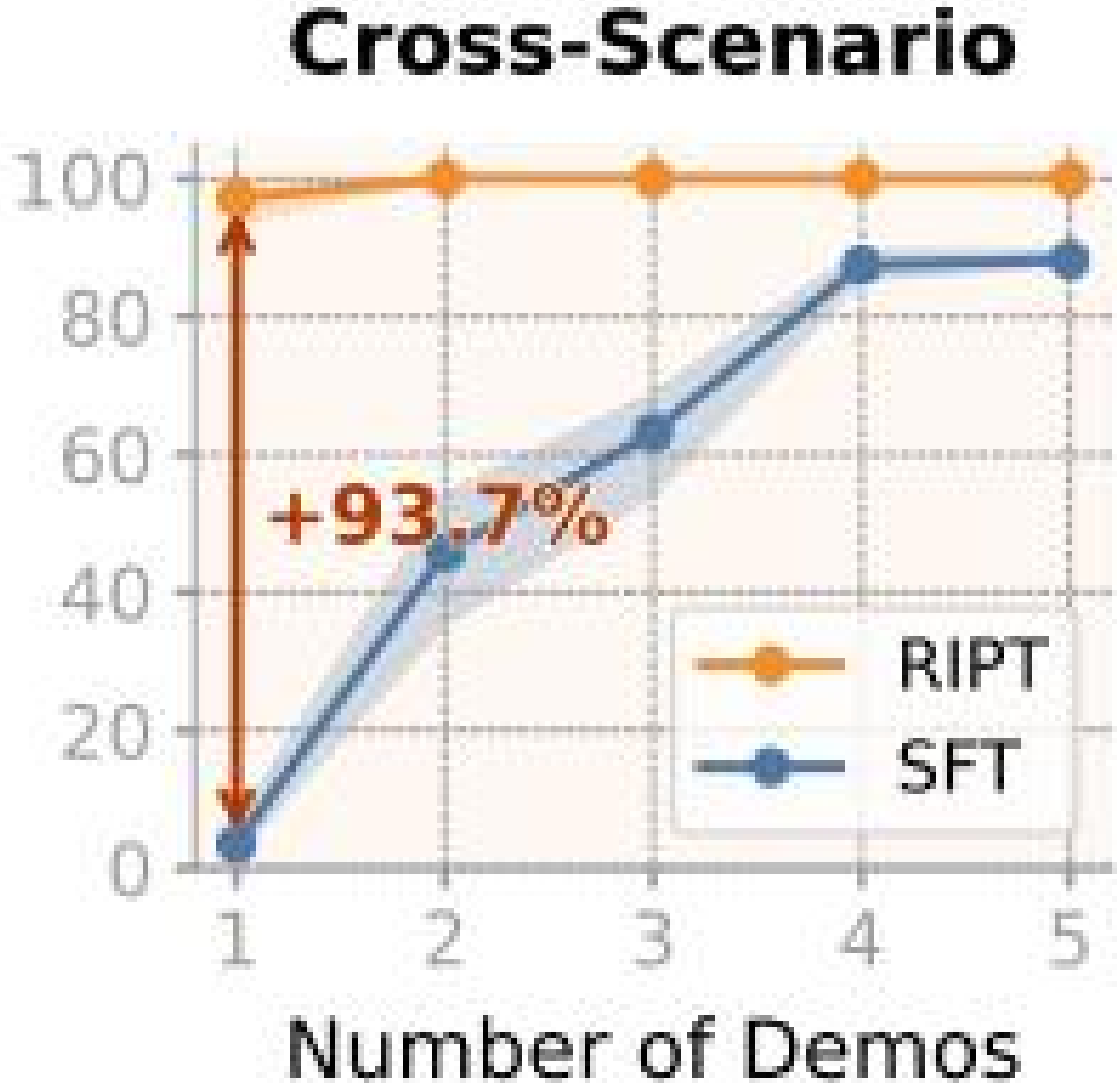


图 1: RIPT-VLA 概览。传统的 VLA 模型通常采用两个监督训练阶段；在此基础上，我们进一步提出第三阶段：面向 VLA 的强化交互式后训练（Reinforcement Interactive Post-Training）。RIPT-VLA 在多个不同基准上取得了 SOTA 结果；同时，在低数据设定下表现出显著优势，仅用一个 demonstration，就能将一个几乎失效的 SFT 模型从接近失败提升到 97% 的成功率。

2 Introduction

Vision-Language-Action (VLA) 模型 [40] 旨在通过统一的接口，使智能体能够在物理世界中完成感知、推理与行动。当前的 VLA 模型通常采用两个监督阶段进行训练：首先在大规模、多样化的人类 demonstration 上进行预训练 (pretraining)，随后在规模较小、面向特定任务的数据集上进行监督微调 (supervised fine-tuning, SFT)。这一范式具有若干显著优势：预训练使 VLA 模型能够形成通用的 visuomotor 技能，而 SFT 则使其能够针对特定环境进一步专门化 [15]。监督训练还使 VLA 模型能够从大规模、预先记录的 vision-language-action 数据集中学习。然而，这种监督式方法也存在两个核心局限。第一，数据采集是离线进行的。VLA 模型学习的是对环境交互的模仿，却从未真正观察到自身动作所带来的后果。因此，所学得的 policy 往往难以应对真实世界场景中的复杂性，尤其是在长时程任务上。第二，基于模仿学习的任务特定 SFT 高度依赖大规模、高质量的人

类 demonstration。这类数据采集成本高、耗时长，而且当 demonstration 数量较少时，模型性能会显著下降。

本文提出 RIPT-VLA，将 Reinforcement Interactive Post-Training 作为 VLA 训练范式的第三阶段。在预训练和监督微调之后，我们让 VLA 模型与多任务环境进行交互，并接收二值的成功/失败奖励。随后，我们通过强化学习直接优化 VLA 模型，使其在多个任务上的成功率 (success rate) 得到提升。受 LLM 推理领域已有 RL 框架 [9] 的启发，我们在第 4 节提出了一种用于 VLA 微调 (fine-tuning) 的稳定且高效的 RL 框架。具体而言，我们扩展了 LOOP 框架 [4]。该框架结合了 REINFORCE leave-one-out (RLOO) 优势估计 [16] 与 proximal policy optimization (PPO) [28]。不同于 LOOP，我们构造由非零优势样本组成的均匀 batch：过滤掉任何优势为零的轨迹组，并持续采样 rollout，直到获得足够数量的有效样本。这样的均匀 batch 构造能够提升训练稳定性，尤其是在训练后期、VLA 模型成功率逐步升高时更为明显。RIPT-VLA 能够在不依赖 shaped reward、学习式奖励或 critic 模型的情况下，实现高效且稳定的 VLA policy 更新。将强化学习作为第三训练阶段还带来若干明显优势：其数据效率更高，即便每个任务仅使用单个 SFT demonstration，也能取得接近 state-of-the-art 的性能；同时，由于模型在训练过程中能够真实观察与环境的交互结果，最终得到的 VLA 模型在终端任务上的表现也显著更强。RIPT-VLA 同时适用于 tokenized action [22] 和连续动作 [14] 两种设定。

RIPT-VLA 与近期 LLM 训练范式转变的趋势高度一致 [24,9]。在大规模文本语料上进行预训练，能够赋予 LLM 广泛的知识和强大的能力，但它们在需要精确推理、多步规划或工具使用的挑战性任务上，往往仍表现不足 [34]。为应对这些局限，强化学习已逐渐成为关键的后训练阶段：仅需少量交互式反馈，便可重新激活并引导预训练知识 [24]。类似地，我们观察到，预训练的 VLA 模型同样编码了丰富的 visuomotor 技能，但在面对新任务和新场景时，往往难以有效调用这些能力。RIPT-VLA 通过仅使用稀疏的二值奖励，并结合少量优化步骤，弥合了这一差距，从而释放出这些潜在技能。

3 1 Introduction (第 2 部分)

在第 5 节的全面实验中，我们表明：RIPT-VLA 与大规模 VLA 模型和轻量级 VLA 模型结合时，都能在多样化任务上取得 SOTA 结果。在 LIBERO 基准测试 [?] 上，RIPT-VLA 将当前最优的轻量级 VLA 模型 QueST [?] 在四个任务套件上的表现全部提升，平均带来 10.9% 的绝对成功率 (success rate, SR) 增益 (表 1)。在 OpenVLA-OFT [?] 上进行评测时，尽管该模型已是表现最好的大型 VLA 模型，且成功率已经达到较高的 96.7%，RIPT-VLA 仍能进一步将失败率从 3.3% 降至 2.5%。我们还在多任务基准 LIBERO-90 (94.3%) 和 MetaWorld45 [?] (92.2%) 上取得了领先结果，这说明 RIPT-VLA 能够有效提升单一模型在多任务（最多 90 个任务）设定下的性能 (表 2)。尤为值得注意的是，在仅有单条训练 demonstration 的极低数据场景下，RIPT-VLA 仍能以极高效率将预训练知识自适应到新的任务目标或场景中，仅需 15 次 RL 迭代，就可将成功率从低于 4% 提升到超过 97%。

4 2 Related Works

Vision-Language-Action Models. Vision-Language-Action (VLA) 模型使具身智能体能够理解多模态输入，如视觉观测与自然语言指令，并将其转化为物理世界中的有效动作 [40]。代表性工作包括 RT-2 [40]、RT-1 [2]、PaLM-E [7]、Octo [32]、Dita [11]、 π_0 [1]、 $\pi_{0.5}$ [13] 以及 OpenVLA [15]。这些工作表明，VLA 在语义推理与对新任务、新环境的泛化方面已经展现出涌现能力。此类

模型通常采用两阶段监督学习范式进行训练：首先在大规模、web-scale 数据集上进行预训练 [7, 3], 以获得可泛化的 visuomotor skill、将语言锚定到感知之中, 并构建稳健的内部表征; 随后再面向具体任务进行监督微调 (fine-tuning)。尽管这一两阶段范式推动了该方向的发展, 其离线训练属性也带来了关键局限。具体而言, 在监督微调 (SFT) 阶段, 面对新的任务或环境时, 模型通常仍需大量专家 demonstration, 这会削弱其 few-shot 适应能力。这表明当前方法仍存在一个重要缺口: 需要能够超越静态模仿、利用交互式经验来适应预训练 VLA, 并减少对大规模专家数据依赖的方法。

Reinforcement Learning for Large Language Models. Large Language Models (LLMs) 为增强预训练模型能力提供了重要先例。尽管 LLMs 可以通过预训练与 SFT 获得广泛能力, 但在复杂推理、规划以及约束满足等问题上仍常常表现不足 [34]。为解决这一问题, 强化学习 (RL) 已成为 LLM 训练中的一个关键“第三阶段”: 它使模型能够基于交互反馈而非静态数据集进行学习 [24]。近期研究表明, RL 能够激发模型在数学求解 [19, 29]、可自验证证明 [20]、基于 tree-of-thoughts 的长时程规划 [35], 以及结合 AI feedback 的偏好对齐生成 [17] 等方面的潜在能力。这一范式的核心在于: 通过有针对性的反馈来引导预训练知识的使用。该思路也为 Vision-Language-Action 模型提供了直接启发: RL 有望使预训练 VLA 更有效地适应具身任务中强交互、强后果约束的本质特征。

Reinforcement Learning for VLA. 近期已有一些工作尝试将强化学习应用于预训练 VLA, 以突破监督微调的局限, 并在不收集新 demonstration 的情况下适应新任务。iRe-VLA [10] 通过在冻结的 VLM backbone 上交替执行基于 PPO 的更新与监督蒸馏阶段, 来缓解优化不稳定问题。然而, 该方法在 PPO 过程中仍依赖学习得到的 value critic, 并且需要设计 shaped reward function 或采用 success weighting 来引导 policy 学习。ConRFT [5] 则进一步将离线 Q-learning 与在线 consistency-policy 更新结合起来, 但同样依赖参数化的 value function。这两类方法都需要精细协调离线与在线阶段, 以稳定 critic 的学习。相比之下, RIPT-VLA 提出了一种完全不依赖 critic 的优化框架, 在稀疏二值奖励下具有更简单的训练动态。

5 预备知识

5.1 3.1 Vision-Language-Action Models (第一部分)

自回归式 VLA rollout. Vision-Language-Action(VLA)模型 π_θ 将观测序列与历史动作 ($o_{1:t}, a_{1:t-1}$), 以及自然语言目标 g , 映射为下一时刻动作 a_t 的概率分布。这类模型以自回归方式运行, 即 $a_t \sim \pi_\theta(\cdot | o_{1:t}, g, a_{1:t-1})$ 。给定初始的观测-目标上下文 $\mathbf{c} = (o_1, g)$, 模型基于过去信息, 以自回归方式生成动作序列:

$$\pi_\theta(a_{1:T} | o_{1:T}, g) = \prod_{t=1}^T \pi_\theta(a_t | o_{1:t}, g, a_{1:t-1}). \quad (1)$$

我们将这一采样过程记为 $\mathbf{a} = a_{1:T} \sim \pi_\theta(\cdot | \mathbf{c})$, 对应的观测序列记为 $\mathbf{o} = o_{1:T}$ 。当任务成功完成或达到时间上限时, 序列终止。对于每条 rollout 序列及其任务目标 g , 环境 $\mathcal{E}$ 会返回二值奖励: 若成功达到任务目标, 则 $R = 1$; 否则 $R = 0$ 。环境 $\mathcal{E}$ 可以是模拟器 [36, 21], 也可以是真实世界。

VLA 模型中常见的动作预测方式主要有两类。其一是 tokenized action head: 将动作表示为来自固定词表的离散 token, 并通过对 token 集合做分类来预测动作。其二是 regression action head: 直接通过回归预测实值动作向量。

当前的 VLA 训练范式. 当前的 Vision-Language-Action (VLA) 模型通常采用两阶段训练：阶段 1 为预训练 (pretraining)，阶段 2 为监督微调 (supervised fine-tuning)。

在阶段 1 中，基础 policy π_θ 会在大规模、多样化的真实世界 demonstration 数据集上进行预训练，记为 $\mathcal{D}_{\text{pretrain}} = \bar{\rho}\{(\mathbf{o}, \mathbf{a}, \bar{\rho}g)\}_{i=1}^N$ 。训练目标是基于 $\mathcal{D}_{\text{pretrain}}$ 中的离线数据，模仿真实动作。对于采用 tokenized action head 的 VLA 模型，其损失为：

$$\mathcal{L}_{\text{pre}}(\theta) = -\mathbb{E}_{(\mathbf{o}, \mathbf{a}, g) \sim \mathcal{D}_{\text{pretrain}}} \left[\sum_{t=1}^T \log \pi_\theta(a_t \mid o_{1:t}, g, a_{1:t-1}) \right], \quad (2)$$

而对于 regression action head， $\mathcal{L}_{\text{pre}}(\theta)$ 通常实现为 MSE 或 L1 损失。该阶段使 VLA 模型能够学习到较强的表征，并获得通用的视觉-运动控制与指令跟随能力。

在阶段 2 中，预训练后的 policy 会在更小规模的多任务数据集上进行监督微调，以提升其在一组目标任务上的表现，记为 $\mathcal{D}_{\text{sft}} = \{(\mathbf{o}, \mathbf{a}, g)\}_{i=1}^{N'}$ 。通常， $\mathcal{D}_{\text{sft}}$ 对每个任务包含约 50 条高质量人工 demonstration [22]。这一阶段仍采用与阶段 1 相同的目标函数进行训练。其作用在于：将阶段 1 中学到的通用技能进一步适配为面向目标任务的一组专门技能。

尽管两阶段流程已成为当前 VLA 训练的标准做法，但它仍存在两个显著问题。首先，该流程仅依赖离线监督，缺乏来自环境的交互式反馈。因此，受分布偏移和误差级联影响，所学得的 policy 在真实 rollout 中往往容易失败，尤其是在长时程 rollout 场景下更为明显。其次，VLA 的性能高度依赖 $\mathcal{D}_{\text{sft}}$ 中任务特定数据的质量和数量，而这类数据通常难以获取，且采集成本较高。

5.2 3.1 Vision-Language-Action Models (第 2 部分)

将 VLA 建模为 Markov 决策过程。为了更好地优化 VLA 模型，我们将其任务定义为一个 Markov 决策过程 (MDP)。每个 episode 在上下文 $\mathbf{c} = (o_1, g)$ 下初始化，其中 o_1 表示初始观测， g 表示语言目标。状态表示为 $[o_{1:t}, g, a_{1:t-1}]$ ，包含语言目标 g 、历史观测序列 $o_{1:t}$ ，以及过去的动作序列 $a_{1:t-1}$ 。在每个时间步 t ，VLA policy 从如下 policy 分布中采样动作：

$$a_t \sim \pi_\theta(\cdot \mid o_{1:t}, g, a_{1:t-1}).$$

随后，环境依据隐藏的环境动力学转移到下一观测 o_{t+1} ，并形成新状态 $[o_{1:t+1}, g, a_{1:t}]$ 。当执行完一段动作序列 $a_{1:T}$ 后，智能体从环境 $\mathcal{E}$ 获得一个二值奖励 $R(\mathbf{c}, \mathbf{a}) \in \{0, 1\}$ ，表示任务成功或失败。VLA 优化的目标本质上是学习一个 policy π_θ ，使其最大化任务成功奖励的期望：

$$R_\theta(\mathbf{c}) = \mathbb{E}_{\mathbf{a} \sim \pi_\theta(\cdot \mid \mathbf{c})} [R(\mathbf{c}, \mathbf{a})]. \quad (3)$$

5.3 3.2 强化 policy 优化 (第 1 部分)

我们考虑如下强化学习设定：智能体与环境 $\mathcal{E}$ 交互，学习一个 policy $\pi_\theta(\mathbf{a} \mid \mathbf{c})$ ，以最大化期望回报： $\mathbb{E}_{\mathbf{c} \sim \mathcal{D}_{\text{context}}, \mathbf{a} \sim \pi_\theta} [R(\mathbf{c}, \mathbf{a})]$ 。其中， $\mathbf{c}$ 表示上下文 (context，例如目标与初始观测)， $\mathbf{a}$ 表示一条轨迹 (例如动作序列)，而 $R(\mathbf{c}, \mathbf{a}) \in \{0, 1\}$ 是环境返回的稀疏二值奖励。为优化这一目标，标准做法是采用 policy gradient，其对 π_θ 的更新形式为：

$$\nabla_\theta L_\theta(\mathbf{c}) = \mathbb{E}_{\mathbf{a} \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(\mathbf{a} \mid \mathbf{c}) \cdot A(\mathbf{c}, \mathbf{a})], \quad (4)$$

其中, $A(\mathbf{c}, \mathbf{a})$ 是 advantage function, 用于衡量动作 $\mathbf{a}$ 相对于某个 baseline 有多好。实际中, $A(\mathbf{c}, \mathbf{a})$ 往往难以计算, 尤其是在稀疏奖励场景下。为解决这一问题, 近期工作提出了一种无 critic 的优化框架, 称为 Leave-One-Out Proximal Policy Optimization (LOOP) [4]。具体来说, 它结合了下面两种方法。

Leave-One-Out Advantage Estimation (RLOO) [16]. 对于每个采样得到的上下文 $\mathbf{c}$, 在固定的采样 policy π_ψ 下, 我们采样 K 条 rollout, 记为 $\{\mathbf{a}_k \sim \pi_\psi(\cdot | \mathbf{c})\}_{k=1}^K$ 。每条 rollout 都会获得一个二值奖励 $\bar{R}_k = R(\mathbf{c}, \mathbf{a}_k)$ 。对第 k 条 rollout, 其 leave-one-out baseline 通过其余 rollout 的奖励均值计算得到:

$$b_k = \frac{1}{K-1} \sum_{j \neq k} R_j, \quad A_k = R_k - b_k. \quad (5)$$

这种按组归一化的 advantage 表示: 在相同上下文下, 一条 rollout 的表现相对于其他 rollout 是更好还是更差。借助这一机制, 即使奖励是稀疏的二值信号, 也能高效构造出稳定的 advantage 信号, 而无需额外学习 value function。

Proximal Policy Optimization (PPO) [28]. 为了利用收集到的 rollout $\{(\mathbf{c}_k, \mathbf{a}_k, A_k)\}$ 更新 π_θ , 我们先计算重要性比率 $r_k = \pi_\theta(\mathbf{a}_k | \mathbf{c}_k) / \pi_\psi(\mathbf{a}_k | \mathbf{c}_k)$, 其中 π_θ 是当前正在更新的 policy, π_ψ 是固定的采样 policy (通常设为 π_θ 的最新 checkpoint)。随后, 采用如下截断目标对 π_θ 进行优化:

$$\mathcal{L}_{\text{PPO}} = -\min(r_i A_i, \text{clip}(r_i, 1 - \epsilon, 1 + \epsilon) A_i), \quad (6)$$

5.4 3.2 强化 policy 优化 (第二部分)

算法 1: RIPT-VLA: 面向 VLA 模型的强化式交互后训练(Reinforcement Interactive Post-Training)

输入: 预训练 VLA π , 奖励函数 $R(\mathbf{c}, \mathbf{a})$, 上下文数据集 $\mathcal{D}_{\text{context}}$

1. 对 step = 1 到 M:
2. 更新用于采样的 VLA: $\pi_\psi \leftarrow \pi_\theta$
3. 初始化空数据集 $\mathcal{D}_{\text{rollout}} \leftarrow \emptyset$
4. 当 $|\mathcal{D}_{\text{rollout}}| < B$ 时执行: ▷ Rollout 收集
5. 从 $\mathcal{D}_{\text{context}}$ 中采样一个上下文 $\mathbf{c} \leftarrow (g, o_1)$
6. 生成 K 个 rollout: $\{\mathbf{a}_k \sim \pi_\psi(\cdot | \mathbf{c})\}_{k=1}^K$ ▷ 组采样 (Group Sampling)
7. 计算奖励: $\{R_k \leftarrow R(\mathbf{c}, \mathbf{a}_k)\}_{k=1}^K$
8. 计算 baseline: $b_k \leftarrow \frac{1}{K-1} \sum_{j \neq k} R_j$ ▷ Leave-One-Out Baseline
9. 对每个 k 计算 advantage: $A_k \leftarrow R_k - b_k$
10. 如果所有 $\bar{A} = 0$, 则 ▷ 动态拒绝 (Dynamic Rejection)
11. continue
12. 结束 if

13. 将所有 k 对应的 $(\mathbf{c}, \mathbf{a}_k, A_k)$ 加入 $\mathcal{D}_{\text{rollout}}$
14. 结束 while
15. 对 iteration = 1 到 N:
16. 在 $\mathcal{D}_{\text{rollout}}$ 上, 使用 PPO loss (式 (6)) 更新 π_θ ▷ policy 优化
17. 结束 for
18. 结束 for

其中, ϵ 是一个较小的更新阈值 (本文取 0.2)。该目标函数会鼓励具有正 advantage 的 rollout, 同时在 π_θ 偏离其先前版本 π_ψ 过远时抑制不稳定更新。

LOOP 采用 PPO 来优化由 RLOO 估计得到的 advantage。这样一来, 在稀疏奖励场景下, 即使不引入 critic, 也能够实现较高样本效率的 policy 优化。作为第 4 节交互式后训练框架的一部分, 它提供了一个开箱即用且实际有效的实现。

6 4 RIPT-VLA

如前所述, 当前的 VLA 训练范式与我们的根本目标之间仍然存在鸿沟: 我们真正关心的是让模型在下游任务中切实发挥作用。一方面, 仅依赖离线数据进行纯监督训练, 会使 policy 在真实 rollout 中较为脆弱, 其原因在于误差会逐步累积, 同时离线数据集与在线 rollout 之间还存在分布差异。进一步地, 为了构建离线数据集, 尤其是 $\mathcal{D}_{\text{sft}}$, 通常需要采集足够数量的高质量 demonstration, 而模型也很容易对训练分布发生过拟合。换言之, 通过公式 2 对 VLA 进行优化, 并不必然能够提升公式 3 所定义的 VLA 任务执行成功率。

为弥合这一差距, 我们提出一种新的 VLA 训练范式: 通过与环境 E 的交互, 直接对预训练 VLA 进行优化。我们将这一方法称为 Reinforcement Interactive Post-Training, 简称 RIPT-VLA。

6.1 4.1 VLA 模型的强化交互式后训练

我们的 VLA 训练范式前两个阶段与标准设定相同。第一阶段中, 我们在大规模且多样的数据集 $\mathcal{D}_{\text{pretrain}}$ 上对 VLA 模型进行预训练 (pretraining), 以学习视觉-语言表征以及通用的 visuomotor 技能。随后在第二阶段, 我们使用一个较小的数据集 $\mathcal{D}_{\text{sft}}$ 对 VLA 进行微调 (fine-tuning), 使其能够遵循指令完成一组目标任务。经过这两个阶段后, 可以得到一个预训练的 VLA policy π_θ , 它在目标任务上已经具有非零的成功率 (success rate), 尽管该成功率可能很低。

在 RIPT-VLA 中, 我们进一步执行第三阶段: 强化交互式后训练 (Reinforcement Interactive Post-Training)。在这一阶段中, 我们假设能够让 π_θ 在环境 $\mathcal{E}$ 中进行 rollout, 并在 $\mathbf{a} \sim \pi_\theta(\cdot | \mathbf{c})$ 的条件下获得一个二值奖励 $R(\mathbf{c}, \mathbf{a}) \in \{0, 1\}$, 其中 $\mathbf{c}$ 表示初始上下文。此外, 我们还使用一个初始上下文数据集 $\mathcal{D}_{\mathbf{c}} = \{(o_1, g)\}$ 来构造模型 rollout 的任务初始化。通常, 我们通过直接从 $\mathcal{D}_{\text{sft}}$ 的序列中提取初始状态来获得 $\mathcal{D}_{\mathbf{c}}$ 。对于每一次优化步骤, 我们在两个子过程之间交替进行: rollout 收集与 policy 优化。

在 rollout 收集阶段, 我们随机采样上下文 $\mathbf{c}_i \sim \mathcal{D}_{\mathbf{c}}$, 并让 π_θ 与环境 $\mathcal{E}$ 交互, 输出一个动作序列 $\mathbf{a}_i$ 。对于每个 rollout, 我们收集其奖励 $R(\mathbf{c}_i, \mathbf{a}_i)$, 并计算其 advantage $A_i = A(\mathbf{c}_i, \mathbf{a}_i)$ 。 A_i 表示模型在生成该 rollout $\mathbf{a}_i$ 时应当被鼓励的强度 ($A > 0$) 或惩罚的强度 ($A < 0$)。随后, 我们将

所有 rollout 及其奖励信号 $(\mathbf{c}_i, \mathbf{a}_i, A_i)$ 加入 rollout 数据集 $\mathcal{D}_{\text{rollout}}$ 中, 直到获得 B 个 rollout 为止: $\mathcal{D}_{\text{rollout}} = \{(\mathbf{c}_i, \mathbf{a}_i, A_i)\}_{i=1}^B$

在 policy 优化阶段, 我们在 $\mathcal{D}_{\text{rollout}}$ 上使用强化学习算法对 π_θ 进行优化, 以最大化式 3 中定义的期望任务成功率, 并执行 N 次迭代。优化完成后, 我们使用更新后的 VLA policy π'_θ 收集新的 rollout, 进入下一轮步骤。该过程持续重复, 直到达到 M 个步骤, 最终得到 policy π_θ^* , 从而完成整个 VLA 训练范式。随后, 我们将 π_θ^* 部署到环境中进行测试。

尽管 RIPT-VLA 的核心思想较为直接, 但它仍面临若干挑战。首先, 每个 rollout 序列只能提供稀疏的二值奖励, 无法获得 shaped reward。若训练一个学习式奖励模型来预测 shaped reward 数值, 在 rollout 数据有限时很容易导致 reward hacking [30]。其次, 由于 VLA 模型运行于长时程、多任务环境中, credit assignment 会变得高度模糊。这会使 value target (例如来自 TD error 的目标) 极其噪声化, 且信息量不足。第三, 要为 VLA 训练一个稳定的 value function, 通常需要一个容量与 VLA 本身相当的模型, 这会显著增加大规模 VLA 模型的 GPU 显存占用与训练成本 [38]。最后, 在多任务环境中, 不同任务上下文的难度差异可能非常显著: 有些上下文会轻易成功, 而另一些则在所有 rollout 中都持续失败。这会导致成功率高度不均衡, 并使 policy gradient 更新变得不稳定。

6.2 4.2 Dynamic-Sampling Leave-One-Out Proximal Policy Optimization

为了以稳定且具备样本效率的方式实现 RIPT-VLA, 我们在算法 1 中提出了一个简单而有效的 policy 优化框架。首先, 我们采用 LOOP (第 3.2 节) 作为整体实现的基础。LOOP 特别适合我们的 VLA 设定: 一方面, rollout 具有长时域特性; 另一方面, 奖励信号稀疏, 因此需要高效的 advantage 估计方法。此外, 针对多任务环境中的 VLA, 我们设计了一种动态 rollout 采样机制, 用于过滤信息量不足的 context, 从而使 policy 优化更加稳定、高效。

用于 RIPT-VLA 的 LOOP. 我们在 rollout 收集和 policy 优化两个阶段都采用 LOOP [4]。在 rollout 收集阶段, 我们执行 RLOO [16] advantage 估计。在这一步中, 使用最新的 policy π_θ 作为采样 policy π_ψ 。给定单个 context $\mathbf{c} \sim \mathcal{D}_c$, 我们在相同 context 下从该 policy 重复采样 K 次, 从而收集 K 条轨迹: $\{\mathbf{a}_k \sim \pi_\psi(\cdot | \bar{c})\}_{k=1}^K$ 。随后, 从环境 $\mathcal{E}$ 中获得对应的奖励 $\{R_k\}_{k=1}^K$ 。对于每个 rollout k , 我们用公式 5 计算其 advantage A_k 。在每个 epoch 中, 我们从 $\mathcal{D}_c$ 采样 B/K 个 context, 并对其进行分组采样, 最终得到包含 B 个 rollout 的 $\mathcal{D}_{\text{rollout}}$ 。

在 policy 优化阶段, 我们使用 PPO [28] 来稳定 policy gradient 更新。对于每个 rollout 样本 $(\mathbf{c}_i, \mathbf{a}_i, A_i) \in \mathcal{D}_{\text{rollout}}$, 可利用公式 6 计算其训练目标 $\mathcal{L}_{\text{PPO}}$ 。在每个 epoch 内, 我们基于收集到的 rollout 数据集 $\mathcal{D}_{\text{rollout}}$, 采用 mini-batch 方式执行 N 步优化更新。当 $N = 1$ 时, 该方法等价于 on-policy RLOO; 当 $N > 1$ 时, 同一批样本会被重复用于额外更新, 因此优化过程变为部分 off-policy。

动态 rollout 采样. VLA 模型通常工作在多任务环境中 [15, 22, 31], 而不同 context 对应的任务难度往往差异很大。一些 context 已经被 VLA 模型很好地解决, 因此在 K 组采样下几乎总是成功; 另一些 context 则由于任务本身过于复杂, 或存在分布差异而持续失败。这两种情况都会导致某个 rollout 组中的所有样本获得完全相同的奖励 (全为 1 或全为 0), 从而在公式 5 中得到全零 advantage。于是, 公式 6 也无法提供有效的梯度信号。若将这些样本加入 $\mathcal{D}_{\text{rollout}}$, 它们在 batch 优化中只会贡献零梯度, 从而稀释真正有意义的学习信号, 并导致梯度更新不稳定。

为了解决这一问题, 我们采用一种简单但有效的动态拒绝策略: 对于任一采样到的 context, 如果其对应的 K 个 rollout 都得到相同奖励, 我们就丢弃该 context, 并从 $\mathcal{D}_{\text{context}}$ 中重新采样一个新

的 context 进行分组采样。随着训练推进、policy 性能不断提升，越来越多的任务 context 会产生一致成功的 rollout。动态拒绝机制会自然过滤掉这些已经解决的 context，使优化过程能够集中在剩余更困难的 context 上。

更重要的是，这种方法使得在整个 $\mathcal{D}_{\text{rollout}}$ 上进行 PPO 损失（公式 6）的 batch 优化时，所有 mini-batch 都具有相同的有效 batch size。我们的实验发现，这一点对于 RIPT-VLA 中稳定的 policy 优化至关重要。

我们将完整的优化流程总结在算法 1 中。

6.3 泛化 RIPT-VLA 到不同的 VLA 模型

RIPT-VLA 同时兼容 VLA 模型中常见的离散动作表征与连续动作表征。它要求 VLA 模型能够在每一步输出关于动作的概率分布，这一能力会在两个关键环节中用到。首先，在 rollout 采集阶段，为了在相同初始化上下文下通过 group sampling 获得多样化的 rollout，我们需要从模型输出分布中随机采样不同动作：

$$a_t \sim \pi_\theta(a_t \mid o_{1:t}, g, a_{1:t-1}), \quad (7)$$

另一方面，为了实现稳定的 policy 优化，我们在式 6 中计算 trust region [formula]，以将 policy 更新约束在原始 policy 附近的小区域内。该形式中的一个关键组成部分，是在两个 policy 下都计算已采样动作序列的 log-probability。对于采样得到的动作序列 $\mathbf{a} = (a_1, \dots, a_T)$ ，我们将其 log-probability 计算为各时间步 log-probability 之和：

$$\log \pi_\theta(\mathbf{a} \mid \mathbf{c}) = \sum_{t=1}^T \log \pi_\theta(a_t \mid a_{<t}, \mathbf{c}). \quad (8)$$

换言之，只要某个 VLA 模型 π_θ 能够让我们从每一步的动作分布中随机采样动作 a_t ，并计算 $\log \pi_\theta(a_t \mid a_{<t}, \mathbf{c})$ ，就可以将 RIPT-VLA 应用于该模型。

Tokenized action head. 对于输出离散动作的 VLA 模型，例如 QueST [22]，动作被预测为来自固定词表的一串离散 token，其中 action head 是一个使用 NLL loss 训练的分类头。因此，对模型分类头输出的 logits 施加 softmax 后，就可以直接得到 $\log \pi_\theta(a_t \mid a_{<t}, \mathbf{c})$ 。同样，我们也可以直接从 softmax 后的分布中采样 action token。

Regression action head. 对于连续动作的 VLA 模型 [14]，动作通常通过 MSE 或 L1 loss 进行回归，这类输出本身不提供 log-probability。为支持 policy gradient 优化，作者为模型扩展了一个轻量级预测头，用于估计动作值的尺度 σ_θ 。在假设原始输出头提供均值 μ_θ 的前提下，可以将该 policy 分别视为因子化 Gaussian（对应 MSE）或 Laplace（对应 L1）分布，并在 $\mathcal{D}_{\text{sft}}$ 上使用式 2 的 NLL loss 对该尺度头训练若干轮。完成后，就可以利用预测得到的 μ_θ 和 σ_θ ，以 closed-form 的方式对动作 a_t 进行采样，并计算 $\log \pi_\theta(a_t \mid a_{<t}, \mathbf{c})$ 。

7 实验

我们在两个广泛使用的 VLA 学习基准上评估 RIPT-VLA：LIBERO [21] 和 MetaWorld [36]。本文考察了以下几种设置：

1. 第 5.2 节中的标准多任务设置（最多 90 个任务）；
2. 第 5.3 节中的 few-shot 设置（1~5 个 demonstration）；
3. 第 5.4 节和第 5.5 节中的跨任务与跨场景设置，用于展示模型在预训练阶段利用先验知识实现快速泛化的能力。

此外，我们还进行了若干分析实验，以考察 RIPT-VLA 的实际表现，包括训练曲线、ablation study，以及其对上下文数据集方差与多样性的敏感性。

7.1 5.1 设置

基准测试 LIBERO [21] 是一个面向终身学习的 benchmark，包含 5 个任务套件（task suite）。每个套件都由一组语言引导的 manipulation 任务组成，覆盖多种物体类型、任务定义以及环境场景。具体而言，它包含 4 个套件：Goal、Spatial、Object 和 Long。每个套件都用于评估物体操纵的某一特定能力，并各自包含 10 个不同任务。此外，LIBERO 还提供了一个 LIBERO-90 套件，包含 90 个不同任务，用于大规模评估多任务性能。MetaWorld [36] 是一个面向小样本学习模型的 manipulation 任务 benchmark。本文采用其中的 Meta-Learning 45（ML45）套件，该套件包含 45 个训练任务和 5 个留出任务。

对于这两个 benchmark，每个任务都提供 50 条专家 demonstration 用于训练。在评估阶段，针对同一套件中的所有任务，统一部署单个 VLA 模型，并在每个任务的 50 个留出测试上下文上执行 rollout。性能指标采用平均任务成功率（success rate, SR）。

表 1: 四个 LIBERO 套件上的多任务 SR (%)。加粗表示最优结果，下划线表示次优结果。由 RIPT-VLA 带来的提升以括号形式标出。*: OpenVLA-OFT 的结果通过运行各套件的官方 checkpoint 获得。

Stage 1 + Stage 2 Models					
Method	Goal	Spatial	Object	Long	Average
Octo [32]	84.6	78.9	85.7	51.1	75.1
OpenVLA [15]	79.2	84.7	88.4	53.7	76.5
Dita [11]	85.4	84.2	96.3	63.8	82.4
$\pi 0$ + FAST [26]	88.6	96.4	96.8	60.2	85.5
$\pi 0$ [1]	95.8	96.8	98.8	85.2	94.2
OpenVLA-OFT* [14]	97.9	97.6	98.4	92.9	96.7
OpenVLA-OFT + RIPT	99.0 (+1.1)	98.6 (+1.0)	98.6 (+0.2)	93.8 (+0.9)	97.5 (+0.8)

Stage-2 Models					
Method	Goal	Spatial	Object	Long	Average
Diffusion Policy [6]	68.3	78.3	92.5	50.5	72.4
Seer [33]				78.7	
MDT [27]	73.5	78.5	87.5	64.8	76.1
MDT+ [27]		95.2	97.8	83.0	
QueST [22]	80.8	87.4	93.6	68.8	82.7
QueST + RIPT	92.7 (+11.9)	95.6 (+8.2)	98.4 (+4.8)	87.5 (+18.7)	93.6 (+10.9)

基础模型 我们在两种具有不同设计选择的预训练 VLA 模型上验证 RIPT-VLA。

OpenVLA-OFT [14] 是 7B OpenVLA 模型 [15] 的一个 Optimized Fine-Tuned 变体。OpenVLA 由一个多模态 backbone 初始化而来，该 backbone 将 Llama-2 7B 语言模型与双视觉 encoder [23, 37] 结合，并在 97 万条机器人操纵 demonstration 上完成预训练。OFT 用连续解码 head 替换了原始的离散 tokenized action decoder，并采用 $L1$ 回归损失进行训练。该架构可视为一类大规模回归式 action VLA。

相比之下，QueST [22] 是一个小规模 tokenized action VLA 模型，参数量为 2000 万。QueST 首先学习一个 VQ-VAE，用于将短时运动片段压缩为离散的 skill codebook；随后，一个 GPT 风格的 transformer 在图像和语言条件下自回归地预测这些 skill token；最后，由一个小型 decoder 将这些 token 还原为连续的关节控制指令。

7.2 5.1 Setup (第 2 部分)

实现细节。对于 OpenVLA-OFT，我们使用各任务套件官方提供的 checkpoint 对模型进行微调 (fine-tuning)。训练在 4 张 NVIDIA RTX A5000 GPU 上进行，每张 GPU 配备 24 GB 显存。我们采用 LoRA [12]，其 rank 设为 32，并设置 $K = 8$ 、 $B = 192(8 \times 24)$ 、 $N = 1$ 、 $\epsilon = 0.1$ ，同时将每张 GPU 的 PPO mini-batch size 设为 4。LoRA 模块的学习率设为 $1e-4$ ，action head 的学习率设为

$1e-5$ 。按照第 4.3 节的设置，在应用 RIPT-VLA 之前，我们先在 $\mathcal{D}_{\text{sft}}$ 上用 NLL loss 从头训练一个小规模的 Laplace scale header，共 500 步；该 header 的架构与 action header 相同。在训练过程中收集 rollout 时，我们根据该 header 预测的 scale，按照 Laplace 分布随机采样动作。评测时，则直接使用原始 action header 预测的均值。

对于 QueST，由于没有提供官方 checkpoint，我们首先按照官方代码和超参数配置，在每个任务套件上从头训练 base model。在多任务设定下，我们使用 3 张 GPU 运行 RIPT-VLA，并设置 $K = 16$ 、 $B = 2880 (16 \times 180)$ 。在单任务设定下，我们使用 1 张 GPU，并设置 $K = 16$ 、 $B = 160$ 。在这两种设定中，我们统一设定 $N = 20$ 、每张 GPU 的 PPO mini-batch size 为 8、学习率为 $1e-6$ ，以及裁剪参数 $\epsilon = 0.2$ 。

7.3 5.2 标准多任务训练

本节在标准多任务基准上评估 RIPT-VLA。对于每个任务套件，我们都使用每个任务的全部 50 条 expert demonstration 作为其 SFT 数据集 $\mathcal{D}_{\text{sft}}$ 。随后，我们在对应任务套件的数据集上对 base model 进行微调 (fine-tuning)，从而得到 RIPT-VLA。

表 1 对比了不同 VLA 模型在 4 个 LIBERO 套件上的多任务性能。我们按照 VLA 的训练范式将结果分为两组。在 “Stage 1 + Stage 2” 这一组中，我们纳入了 5 个当前最先进的大规模 VLA 模型：Octo [32]、OpenVLA [15]、Dita [11]、 π_0 [1] 以及 OpenVLA-OFT [14]。这类模型的参数规模通常超过 500M，并且会先在大规模通用数据集上进行预训练 (Stage-1)，例如 Open-X Embodiment [25]，再针对每个 LIBERO 套件，使用每个任务 50 条 demonstration 进行微调 (Stage-2)。相比之下，“Stage 2” 这一组包含 4 个具有代表性的小模型：Diffusion Policy [6]、Seer [33]、MDT [27] 和 QueST [22]。这些模型的参数规模均在 50M 以内，并且直接在各个 LIBERO 套件上从头训练。

我们发现，RIPT-VLA 能够显著提升这两类模型中最优 VLA 模型的性能，并在 4 个 LIBERO 套件上刷新 SOTA。具体而言，RIPT-VLA 在 4 个任务套件上将 QueST 的成功率 (success rate) 平均绝对提升了 10.9，其中在更具挑战性的 LONG 套件上，绝对提升更是达到 18.7。值得注意的是，在引入 RIPT-VLA 后，小规模 20M QueST 模型不仅显著优于 Dita (334M) 等大模型，而且其性能也已可与 π_0 (2B) 相当。另一方面，当 RIPT-VLA 应用于当前表现最好的大规模 VLA 模型 OpenVLA-OFT 时，尽管其原始成功率已经很高，RIPT-VLA 仍可进一步将平均失败率从 3.3% 降至 2.4%。借助 RIPT-VLA，我们在 4 个 LIBERO 套件中的 3 个上达到了新的 SOTA (Object 套件仅差 0.2)，并取得了所有任务上的最高平均成功率。这些结果表明，RIPT-VLA 具有很强的普适性：它既能挖掘小规模模型中尚未释放的潜在能力，也能进一步突破高性能模型的上限。

此外，在表 2 左侧两列中，我们给出了 LIBERO-90 和 ML45 上的结果，它们分别包含 90 个和 45 个多样化任务。这两个基准主要用于评估单个 VLA 模型在大量技能上的可扩展性与泛化能力。我们将 RIPT-VLA 应用于 QueST，并与若干具有代表性的模仿学习方法进行比较，包括 ACT [8]、PRISE [39]、Diffusion Policy [6]、VQ-BeT [18] 和 ResNet-T [22]。结果显示，RIPT-VLA 使 QueST 在 LIBERO-90 和 ML45 上的成功率分别获得了 5.7 和 1.2 的绝对提升，再次在这两个基准上达到新的 SOTA。这进一步说明，RIPT-VLA 的价值不仅体现在提升少量相关任务上的性能，也体现在更大规模、更贴近真实应用的场景中，即由单个模型统一解决多种不同任务。

表 2: 4 种评测设置下的平均成功率 (SR%): LIBERO-90 与 ML45 (Full Data), LONG 与 ML45 (5-shot Data)。

Method	Full Data		5-shot Data	
	LIBERO-90	ML45	LONG	ML45
ACT [8]	50.8	90.8	42.0	70.8
PRISE [39]	54.4	80.4	52.7	66.8
DP [6]	75.4	90.3	45.9	65.0
VQ-BeT [18]	81.3	87.6	41.8	65.6
ResNet-T [22]	84.4	88.4	51.9	54.0
QueST [22]	88.6	91.0	50.2	63.6
QueST + RIPT	94.3	92.2	71.4	76.0
(improvement)	(+5.7)	(+1.2)	(+21.2)	(+12.4)

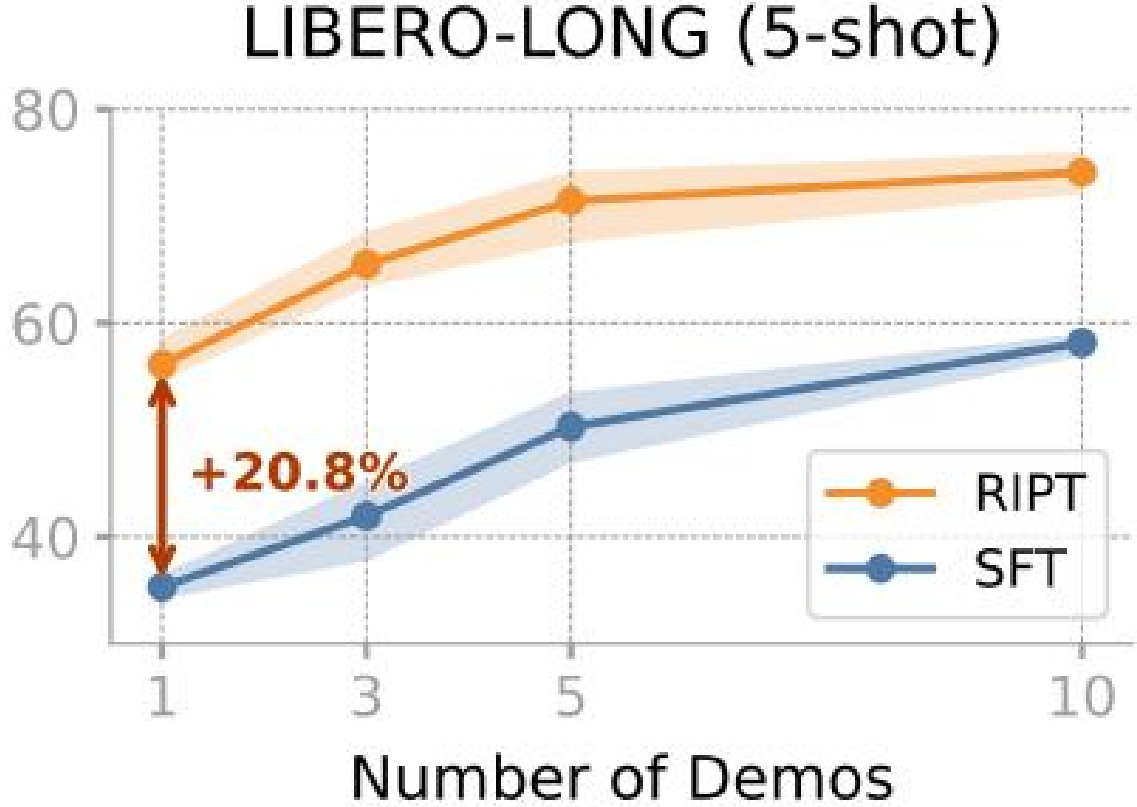


图 2: LIBERO-LONG 上的 few-shot 曲线。

7.4 5.3 少样本多任务训练

本节在少样本多任务设定下评估 RIPT-VLA。对于每个任务套件，我们从每个任务中均匀采样 1 到 10 条 expert demonstration，构成少样本 SFT 数据集 $\mathcal{D}_{\text{sft}}$ 。该设定反映了实际应用中的常见情形，即无法获得大规模数据采集。

表 2 右侧两列给出了 5-shot 设定下的结果，其中 LIBERO-LONG 和 ML45 套件中的每个任务都仅使用 5 条 demonstration 进行训练。基线模型在这一低数据条件下表现较弱，而 RIPT-VLA 在 LIBERO-LONG 上将 QueST 显著提升了 21.2，在 ML45 上提升了 12.4。上述结果表明，RIPT-VLA 有效缓解了标准 VLA 采用 SFT 训练时的一个关键局限：即使只有极少量 demonstration，模型仍能保持较强性能，从而减轻了真实多任务部署中对数据稀缺的担忧。

为进一步分析少样本 demonstration 数量的影响，我们基于 QueST 在 LIBERO-LONG 上进行了不同 few-shot 设定的实验，每个任务的 demonstration 数量从 1 变化到 10。如图 2 所示，在所有数据规模下，RIPT-VLA 都能相对于标准 SFT 模型带来持续且显著的性能提升。值得注意的是，即使在极端低数据条件下——每个任务只有 1 条 demonstration——RIPT-VLA 仍可取得 20.8 的绝对增益。随着 demonstration 数量增加，RIPT-VLA 依然能够继续带来性能改进，体现出很强的样本效率和可扩展性。这些结果进一步验证了 RIPT-VLA 在不同数据稀缺程度下都具有良好的鲁棒性，既适用于低数据场景，也适用于高数据场景。

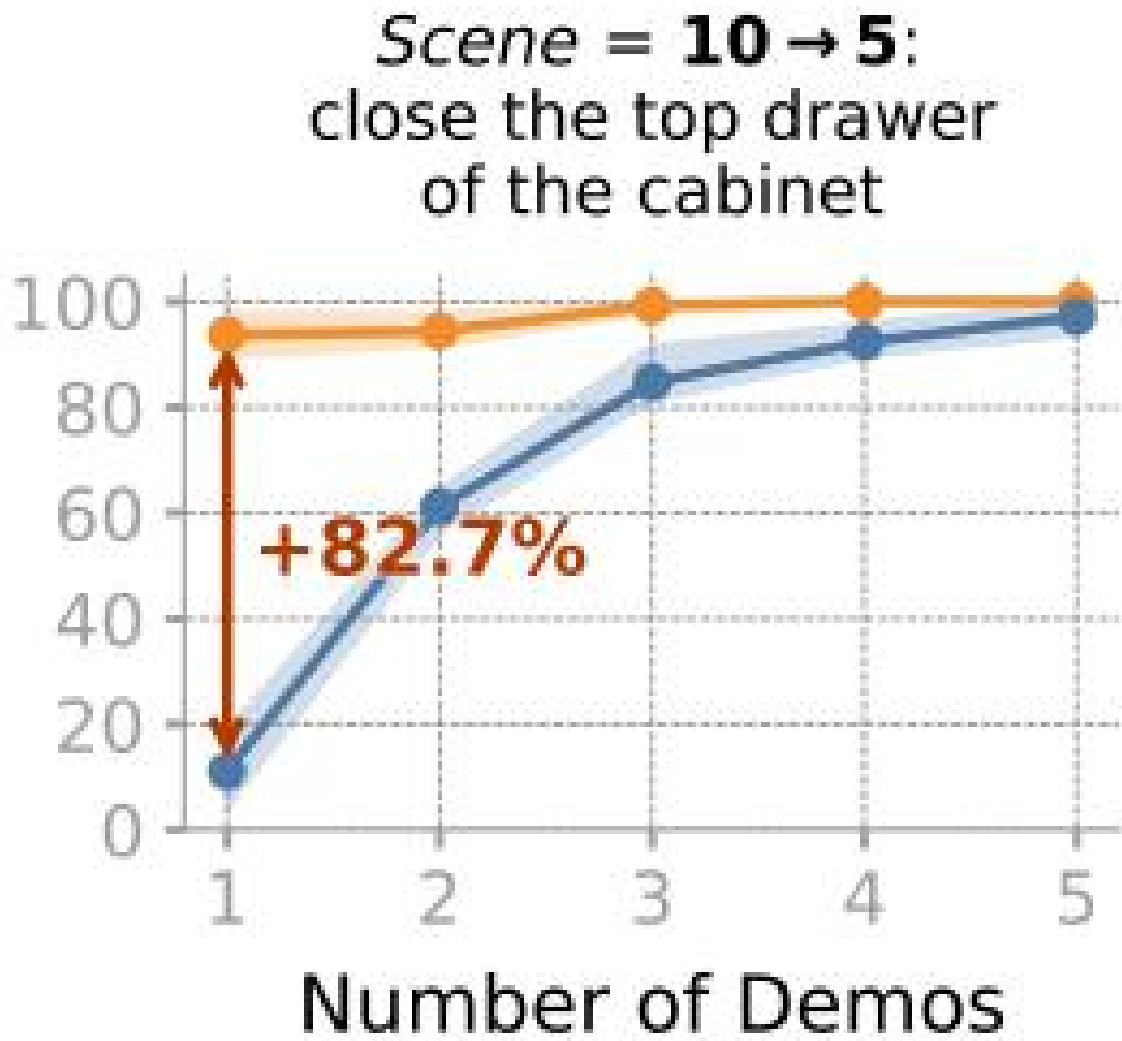


图 3: 跨场景任务泛化示例: 在目标相同的条件下, 从 Scenario A 泛化到 Scenario B。

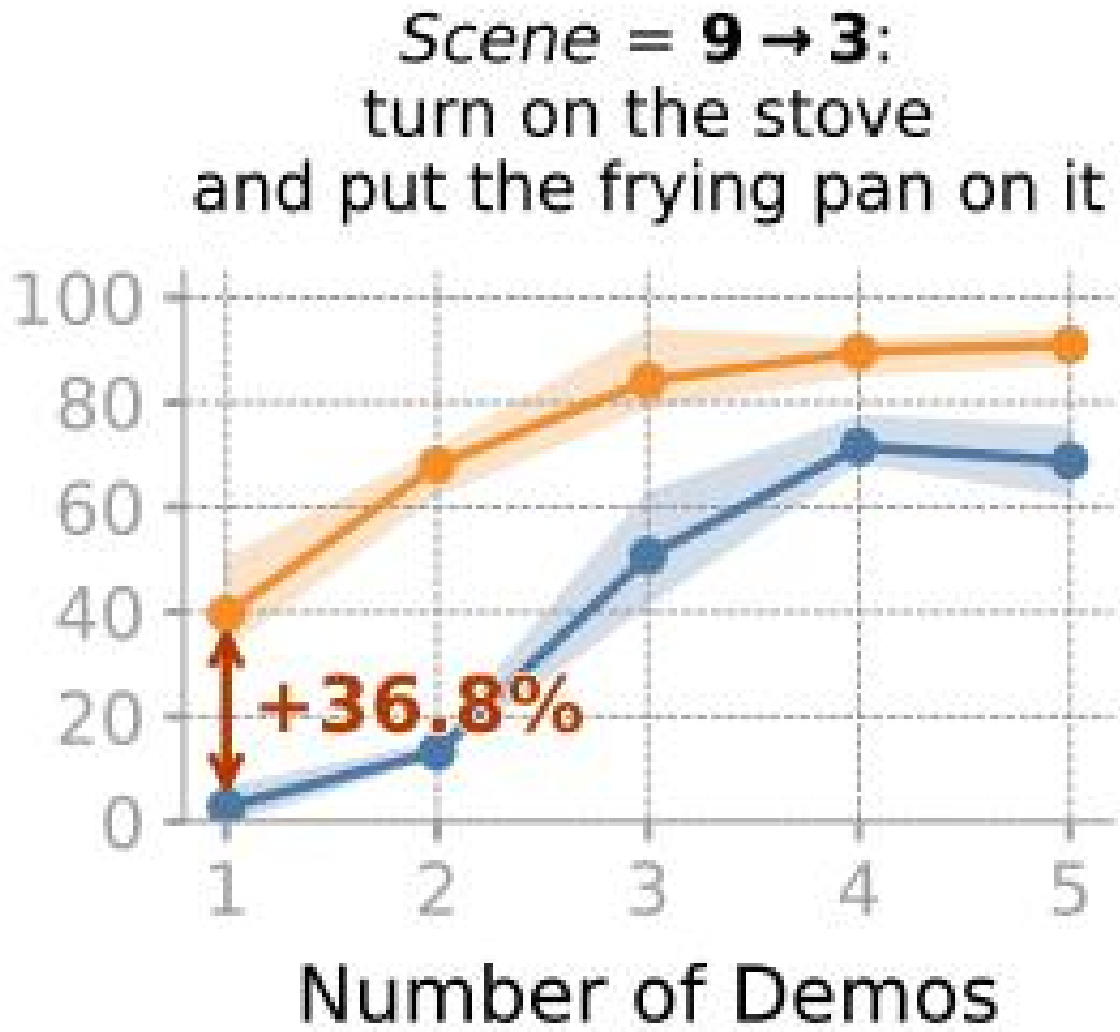


图 4: 跨场景任务泛化示例: 在目标相同的条件下, 从 Scenario A 泛化到 Scenario B。

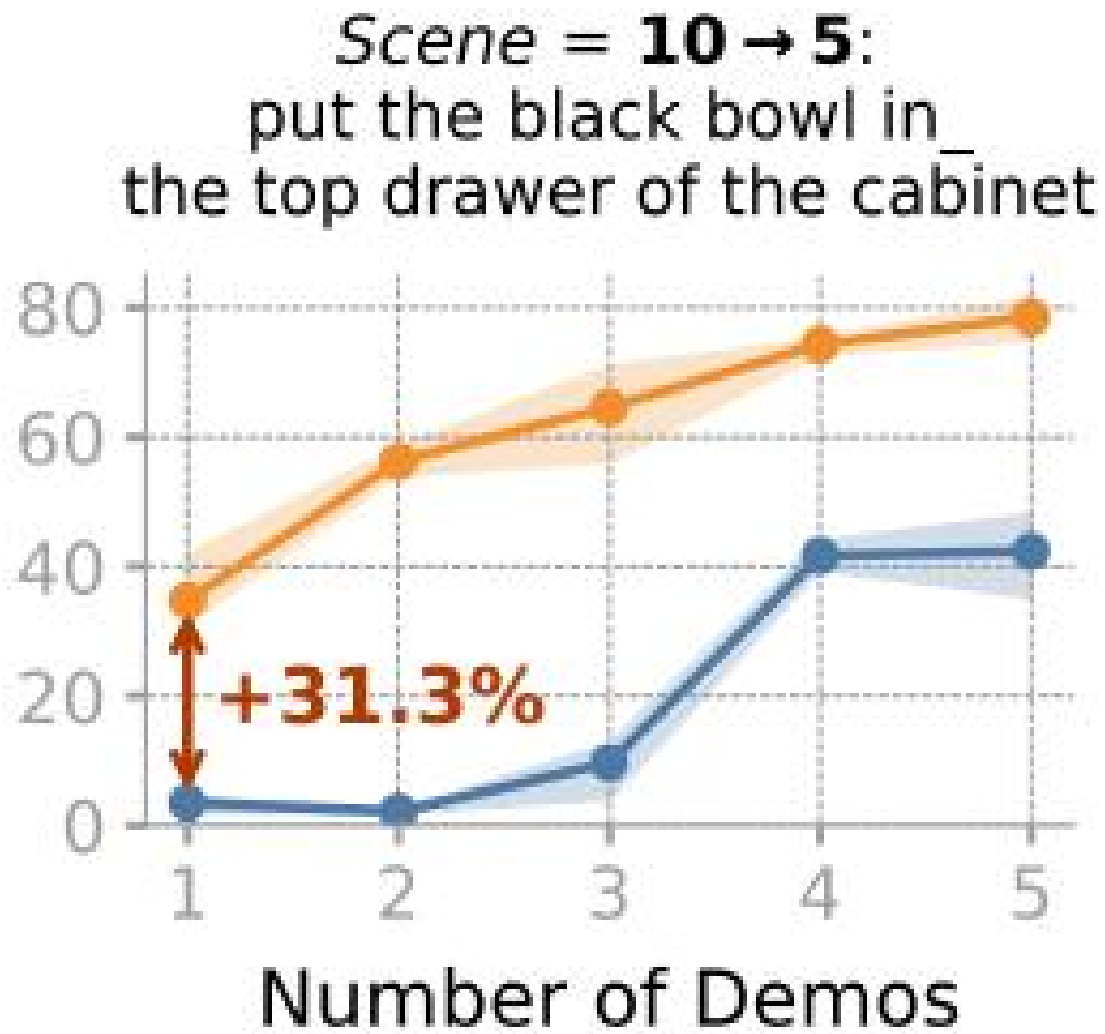


图 5: 跨场景任务泛化示例: 在目标相同的条件下, 从 Scenario A 泛化到 Scenario B。

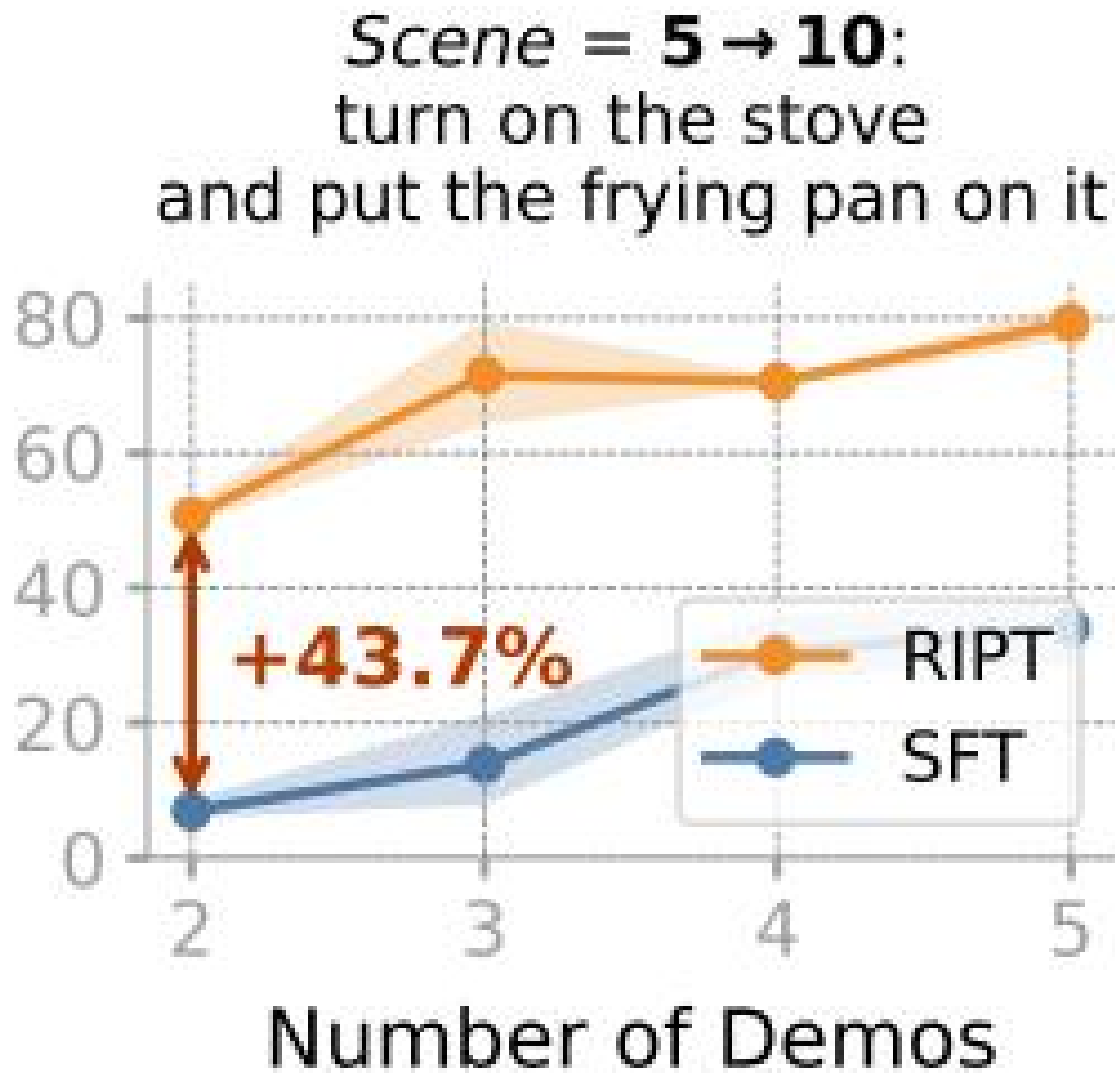


图 6: 图 3: 跨场景任务泛化。在目标相同的条件下, 从 Scenario A 泛化到 Scenario B。

Goal = **Back** → **Front**:
put the butter at the {*Goal*}
in the top drawer of
the cabinet and close it

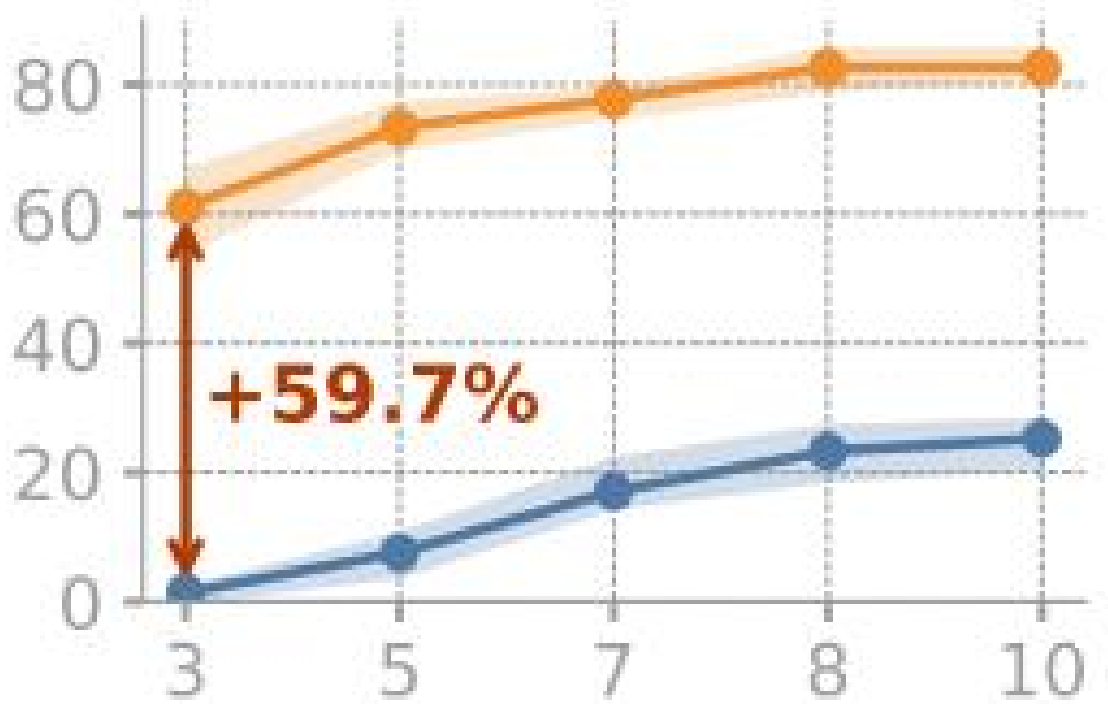


图 7: 跨目标任务泛化示例: 在相同场景中, 从 Goal A 泛化到 Goal B。

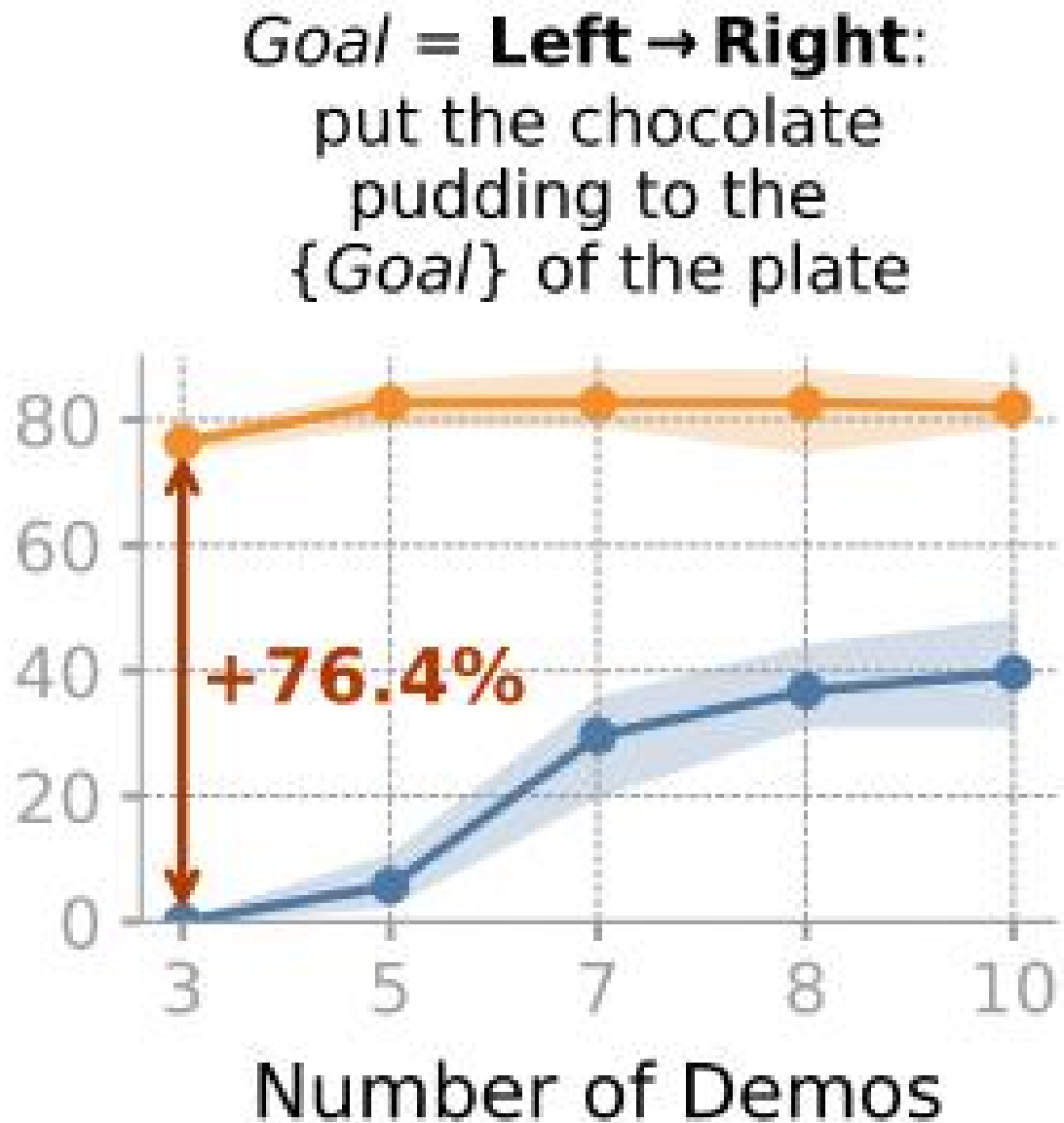


图 8: 跨目标任务泛化示例: 在相同场景中, 从 Goal A 泛化到 Goal B。

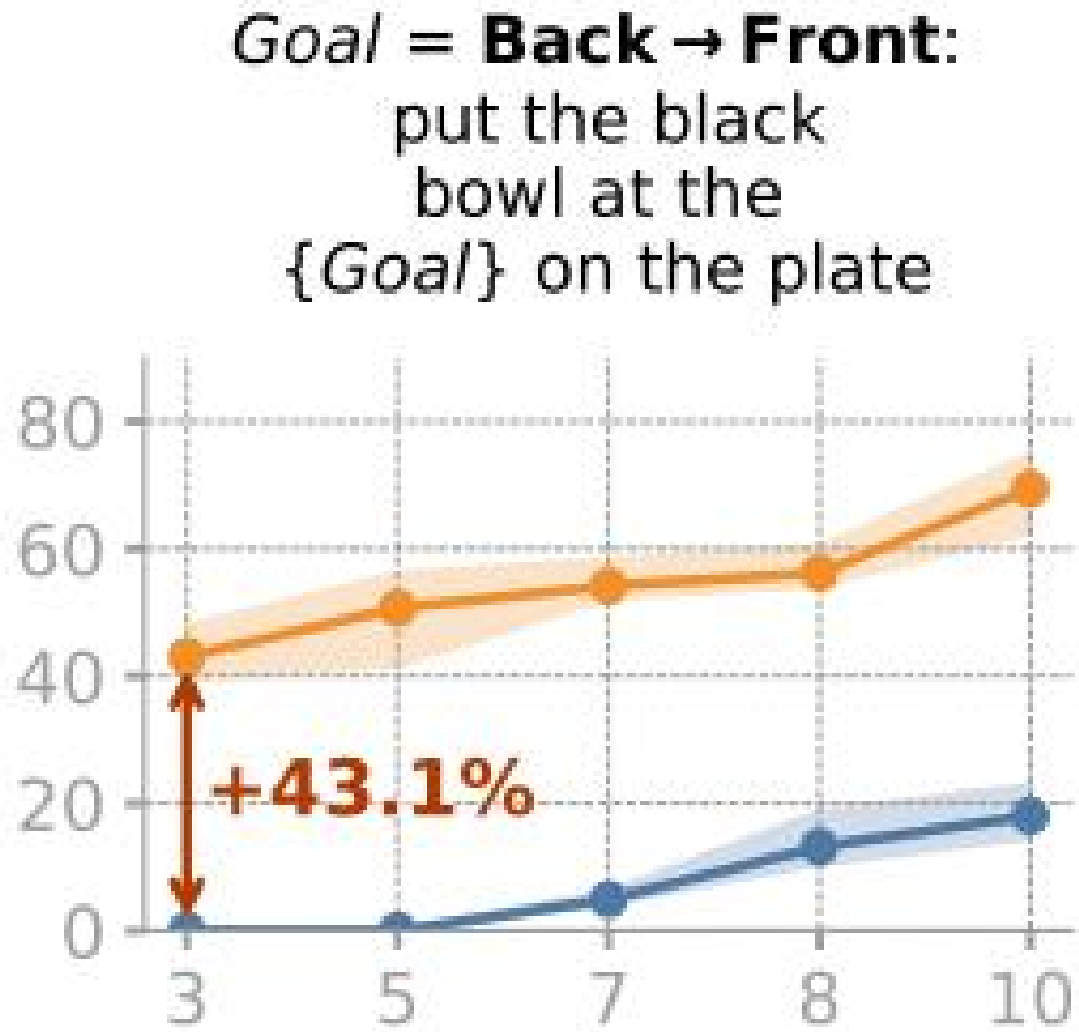


图 9: Goal = Bottom Top: 打开柜体中 {Goal} 对应的抽屉。



图 10: 图 4: 跨目标任务泛化。在相同场景中，从 Goal A 泛化到 Goal B。

7.5 5.4 跨场景泛化

近期 LLM 训练范式的转变表明，只需少量交互反馈，强化学习就能够重新激活并引导预训练知识 [24]。我们在 VLA 中采用了类似思路，并进一步追问：RIPT-VLA 是否能够实现跨场景、跨目标的预训练 visuomotor 技能迁移，同时具备较高的样本效率？

本节在 few-shot 跨场景泛化设定下开展实验。对于每组实验，我们考虑一对任务：二者具有相同的任务目标（例如“打开炉灶并将煎锅放到上面”），但执行场景不同，即场景 A 与场景 B；二者在背景布局和物体配置上均存在明显差异。第 1 阶段中，我们使用来自场景 A 的 $|\mathcal{D}_{\text{pretrain}}| = 50$ 条 demonstration 对 QueST 进行预训练，以获得面向该任务目标的一般性 visuomotor 技能。第 2 阶段中，我们在场景 B 的 $|\mathcal{D}_{\text{sft}}| = \{1, 2, 3, 4, 5\}$ 条 demonstration 上进行 SFT。第 3 阶段中，我们应用 RIPT-VLA，在从 $\mathcal{D}_{\text{sft}}$ 中提取的上下文 $\mathcal{D}_{\text{context}}$ 上通过交互式 rollout 对 policy 进行优化。随后，我们在场景 B 的 50 个测试上下文上评估模型表现。实验使用 3 个随机种子，并绘制了不同 $\mathcal{D}_{\text{sft}}$ 规模下结果的均值与方差。

图 3 给出了 5 组场景对上的实验结果。我们观察到，标准 SFT 在 VLA 模型上的表现，在 1-shot 设定下明显受限：其平均成功率（success rate）仅约为 5%，部分情况下甚至低至 2%。这清楚地说明，SFT 无法将预训练阶段获得的任务知识有效泛化到新场景。相比之下，RIPT-VLA 显著提升了性能，成功率（SR）的绝对增益最高可达 93.7%（即从 SFT 的 3.5% 提升到 97.2%）。随着 $\mathcal{D}_{\text{sft}}$ 规模增大，SFT 与 RIPT-VLA 的性能都会提升，但 RIPT-VLA 始终保持稳定而显著的优势，并且通常仅需 3–5 条 demonstration 就能达到接近 100% 的性能。这些结果支持了我们的核心假设：RIPT-VLA 能够使预训练 VLA 模型在稀疏二值奖励反馈下，快速激活并自适应已学习到的技能。

7.6 5.5 Cross-goal Generalization

在本节中，我们考察 RIPT-VLA 在 cross-goal generalization 设定下的表现。这里我们关注这样一类任务对：它们处于相同场景中，但目标不同。更具体地说，我们选择任务 A 和任务 B，使二者需要相同的 visuomotor 技能，但任务目标不同。例如，任务 A 是“将红色杯子放到右侧盘子上”，而任务 B 是“将红色杯子放到左侧盘子上”。这一设定用于检验：预训练得到的 visuomotor 原语技能（例如抓取与移动）能否被复用并重新组合，从而解决新的任务目标（例如“左”与“右”）所带来的差异。我们仍然遵循 3-Stage 范式：首先在任务 A 的 50 条 demonstration 上预训练 QueST；随后在任务 B 的 3–10 条 demonstration 上进行 SFT；最后将 RIPT-VLA 应用于任务 B。

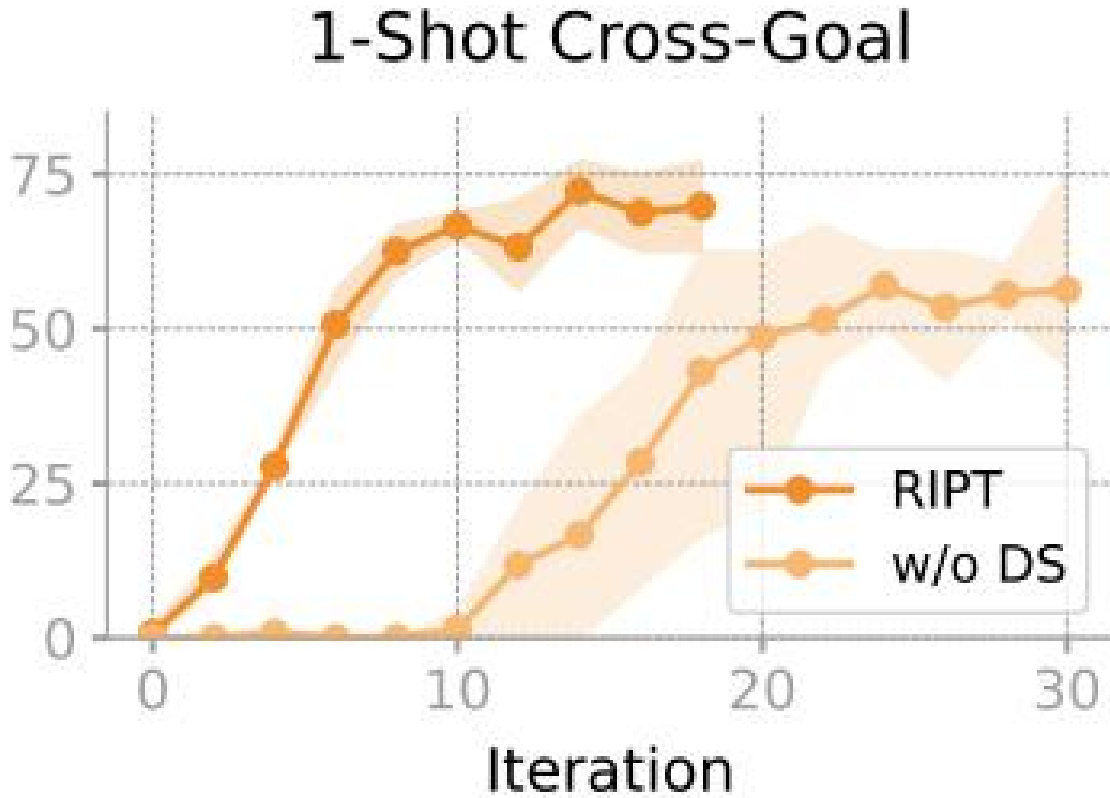


图 11: 动态采样（dynamic sampling）的训练曲线分析。

1-Shot Cross-Scenario

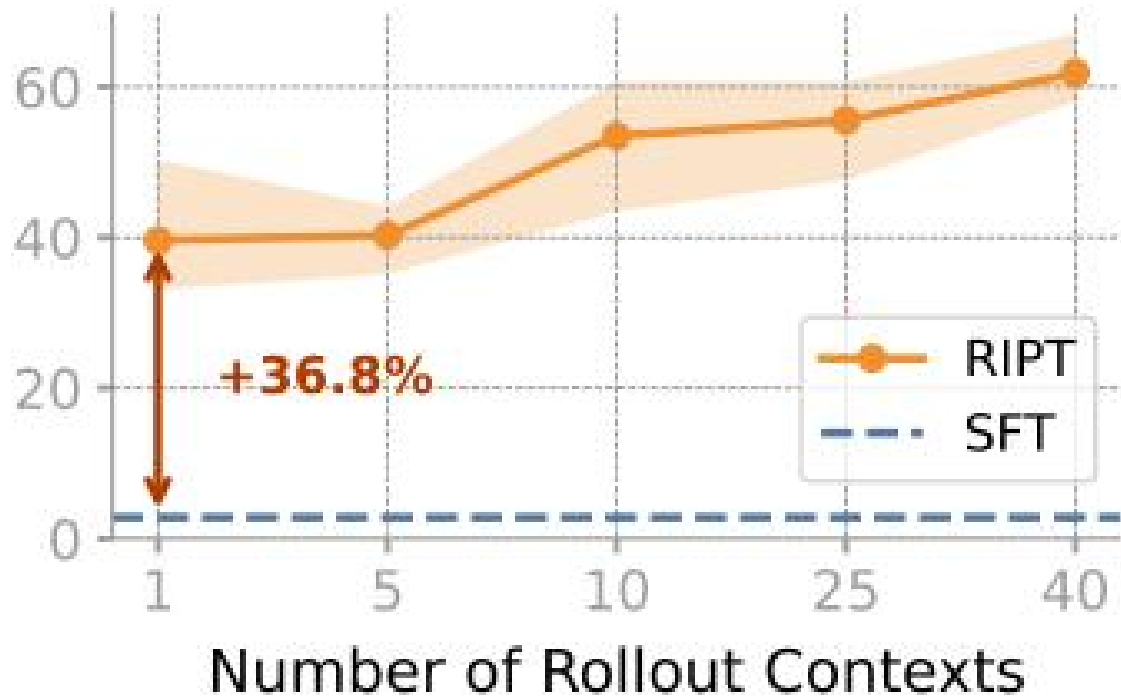


图 12: 上下文数据集规模的分析。

LIBERO-LONG (1-shot)

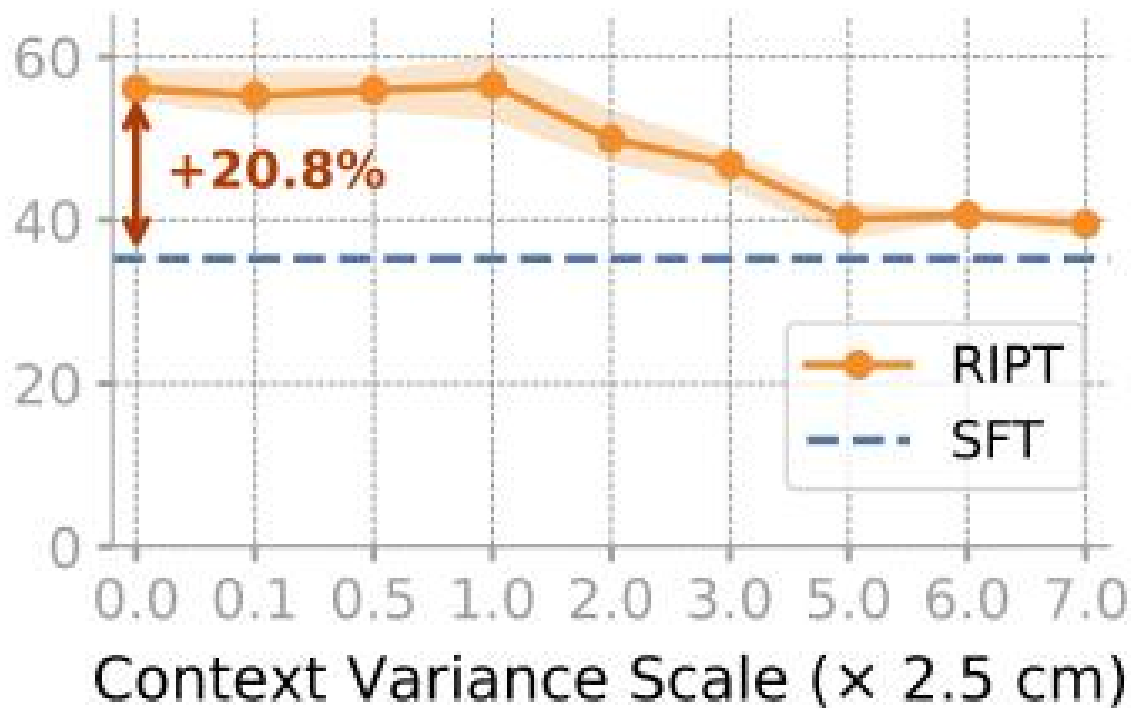


图 13: 初始状态 std 缩放系数的分析。

表 3: 动态采样的消融实验。我们比较了完整 RIPT-VLA、去除动态采样的变体，以及 QueST baseline 在不同任务类型和多任务套件上的表现。

Method	Goal	Spatial	Object	Long	90	ML45	Average
QueST	80.8	87.4	93.6	68.8	88.6	91.0	85.0
+ RIPT-VLA w/o Dynamic Sampling	90.6	91.3	97.5	78.3	92.2	91.3	90.2
+ RIPT-VLA (Ours)	92.7	95.6	98.4	87.5	94.3	92.2	93.5

图 4 给出了 5 组任务上的实验结果。我们观察到, cross-goal generalization 明显更具挑战性。当 demonstration 数量仅为 3 时, SFT 模型仍然难以奏效, 平均成功率 (success rate) 只有 0.7%, 几乎不可用。而引入 RIPT-VLA 后, 模型平均性能可提升至 59.7%。值得注意的是, 对于其中一组任务对, RIPT-VLA 将成功率从接近 0% 大幅提升到 84.7%。随着 demonstration 数量增加, RIPT-VLA 在所有数据区间内都持续保持显著优势。当 demonstration 数量达到 10 时, RIPT-VLA 的平均成功率达到 79.7%, 而 SFT 仅为 29.4%。

这些结果进一步表明, 在低数据 regime 下, SFT 范式在 VLA 泛化方面存在明显局限。相比之下, 我们的方法表明, RIPT-VLA 不仅能够帮助模型将预训练技能适配到新环境中, 还在任务目标语义的快速泛化方面表现突出。

7.7 5.6 附加研究

动态 rollout sampling 的作用。 我们消融分析了第 4.2 节所述动态 rollout sampling 策略的影响。具体来说, 我们将完整的 RIPT-VLA 方法与一个关闭动态 rejection 的变体进行比较。表 3 的结果表明, 动态 sampling 在所有任务类别和任务套件上都能显著提升性能。通过滤除信息量不足的 rollout group, 动态 sampling 能在不同 batch 间提供更一致的梯度信号, 从而保证训练过程稳定且高效。平均来看, 相比不使用动态策略的变体, 成功率 (success rate) 绝对提升了 +3.3, 这说明该策略对于稳定 RIPT-VLA 的训练至关重要。图 5 展示了图 4 第 2 列对应实验的训练曲线 (对 3 个随机种子取平均)。可以看到, 动态 rollout sampling 能加速 RIPT-VLA 的收敛, 使其在整个训练过程中持续取得更高性能, 并表现出更稳定的优化过程。

上下文数据集规模的作用。 为研究上下文数据集 $\mathcal{D}_c$ 的规模如何影响性能, 我们固定使用图 3 第 2 列中、仅基于 1 条 demonstration 进行 SFT 训练的 QueST 模型, 并改变 RIPT-VLA 阶段所使用的 rollout context 数量。图 6 表明, 随着 rollout context 数量增加, 性能会显著提升。这是因为更多的 context 为 rollout 交互提供了更丰富的初始状态多样性, 使模型能够更好地泛化到测试环境中的不同场景设置。值得注意的是, 扩展 $\mathcal{D}_c$ 不需要任何额外的人工标注: 每个 context 仅包含初始观测状态, 无需动作标注。因此, 扩大上下文数据集规模是一种提升 RIPT-VLA 泛化能力的高性价比方式。

RLOO group 中上下文方差的作用。 在公式 5 中, 每个 batch 的 rollouts 会按照共享的初始状态 context 进行分组。但在真实部署中, 由于不可避免的场景布置噪声, 完全一致的初始状态实际上难以实现。为模拟这一情况, 我们统计了 LIBERO-LONG 中物体初始位置的标准差, 其数值约为 2.5 cm。在一个基于 1 条 demo 进行 SFT 的 QueST 模型上, 我们向初始状态中注入不同尺度标准差的高斯噪声, 并运行 RIPT-VLA。图 7 表明, 当方差增大到 $1.0 \times (2.5 \text{ cm})$ 时, 性能仍然保持稳定; 只

有超过 $2.0\times$ 后，性能才开始下降。更值得注意的是，即使在 $7.0\times$ 方差（17.5 cm）下，RIPT-VLA 的表现仍显著优于 SFT baseline。

8 6 结论

本文提出了 RIPT-VLA：一种简单而强大的强化学习范式，用于在稀疏二值任务奖励的监督下，对预训练的 Vision-Language-Action (VLA) 模型进行 post-training。RIPT-VLA 无需设计 shaped reward、价值函数或奖励建模，即可实现稳定且数据高效的优化。该方法在多个 VLA benchmark 上显著提升了性能，并且即使在极低数据量设置下，仍表现出很强的适应能力。RIPT-VLA 可作为一种可扩展的第三阶段训练范式，与现有的预训练和监督微调（fine-tuning）流程形成互补，通过与环境的直接交互释放大规模 VLA 模型的潜在能力。一个值得期待的未来方向，是将 RIPT-VLA 与 VLA 模型中的推理和规划能力相结合，从而在复杂环境中实现更复杂、泛化性更强的行为。

9 参考文献

参考文献条目较多，此处保留原论文英文文献列表，请参阅原文 PDF 的 References 一节。